

University of Engineering and Technology

New Campus, Lahore

Department of Computer Science



FoodExpress Dispatch Optimization Engine

UML / Design Documentation

Submitted by:

Mirza Zain Ali Baig 2025(S)-SE-20

Muhammad Hamza Shahid 2025(S)-SE-08

Muhammad Sohaib Aslam 2025(S)-SE-13

Submitted to:

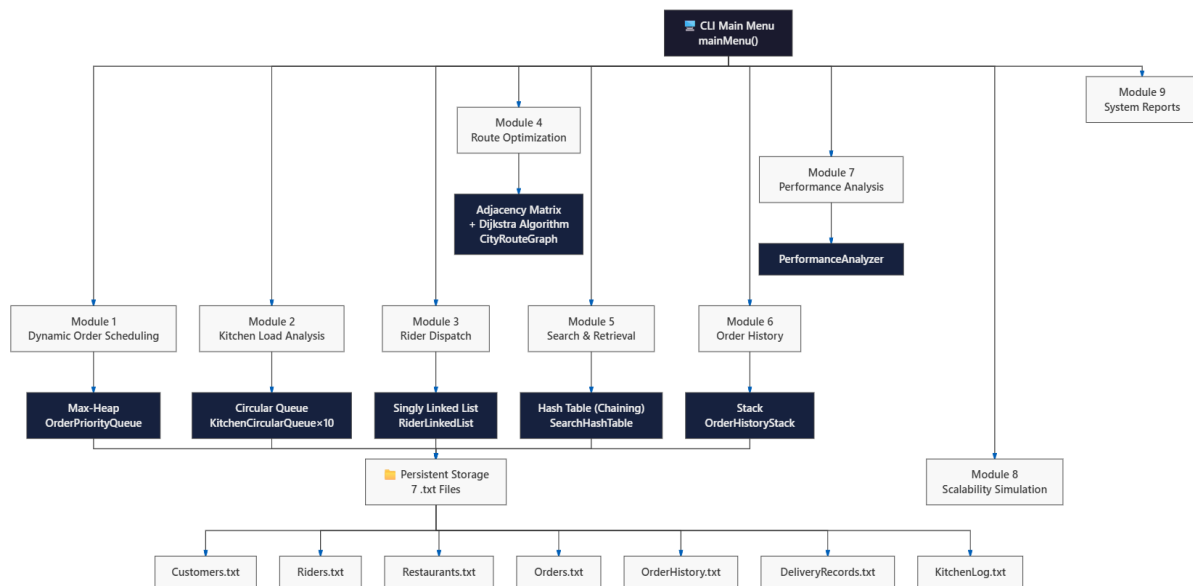
Sir Ali Raza

Table of Contents

1. System Architecture Overview	3
1.1 Architecture Layers	3
1.2 Module-to-Data-Structure Mapping	4
2. Class Diagram & Relationships	4
2.1 Entity Classes	5
2.2 Data Structure Classes	6
2.3 Class Relationships	7
3. Module Design	8
3.1 Module 1 — Dynamic Order Scheduling Engine	8
3.2 Module 2 — Kitchen Load Balancing Module	8
3.3 Module 3 — Rider Dispatch & Assignment Optimization	8
3.4 Module 4 — Real-Time Route Optimization	9
3.5 Module 5 — Search & Retrieval Engine	9
3.6 Module 6 — Order History & Tracking Module	10
3.7 Module 7 — Performance Analysis Module	10
4. Algorithm Flowcharts	11
4.1 Max-Heap Insertion (bubbleUp)	11
4.2 Max-Heap Extract Highest Priority (bubbleDown)	12
4.3 Dijkstra Shortest Path	13
4.4 Hash Table Insert with Chaining	14
4.5 Order Status Undo Operation	15
4.6 Rider Best-Fit Selection	16
5. Complexity Analysis	17
5.1 Time Complexity	17
5.2 Space Complexity	17
6. Ethical & Professional Practices	18
6.1 Customer Data Privacy	18
6.2 Responsible Rider Workload Management	18
6.3 Scalability Limits & Production Considerations	18
6.4 Algorithm Design Justification	19
6.5 Professional Engineering Standards	19

1. System Architecture Overview

The FoodExpress Dispatch Optimization Engine is a CLI-based C++ application structured as a single-executable system with a unified Main Menu that acts as a central dispatcher to 9 functional modules. All modules share a single FoodExpressSystem object (global pointer) which owns the entity collections and all six core data structures. Persistent storage is handled via seven plain-text .txt files, read at startup and written incrementally as operations occur.



1.1 Architecture Layers

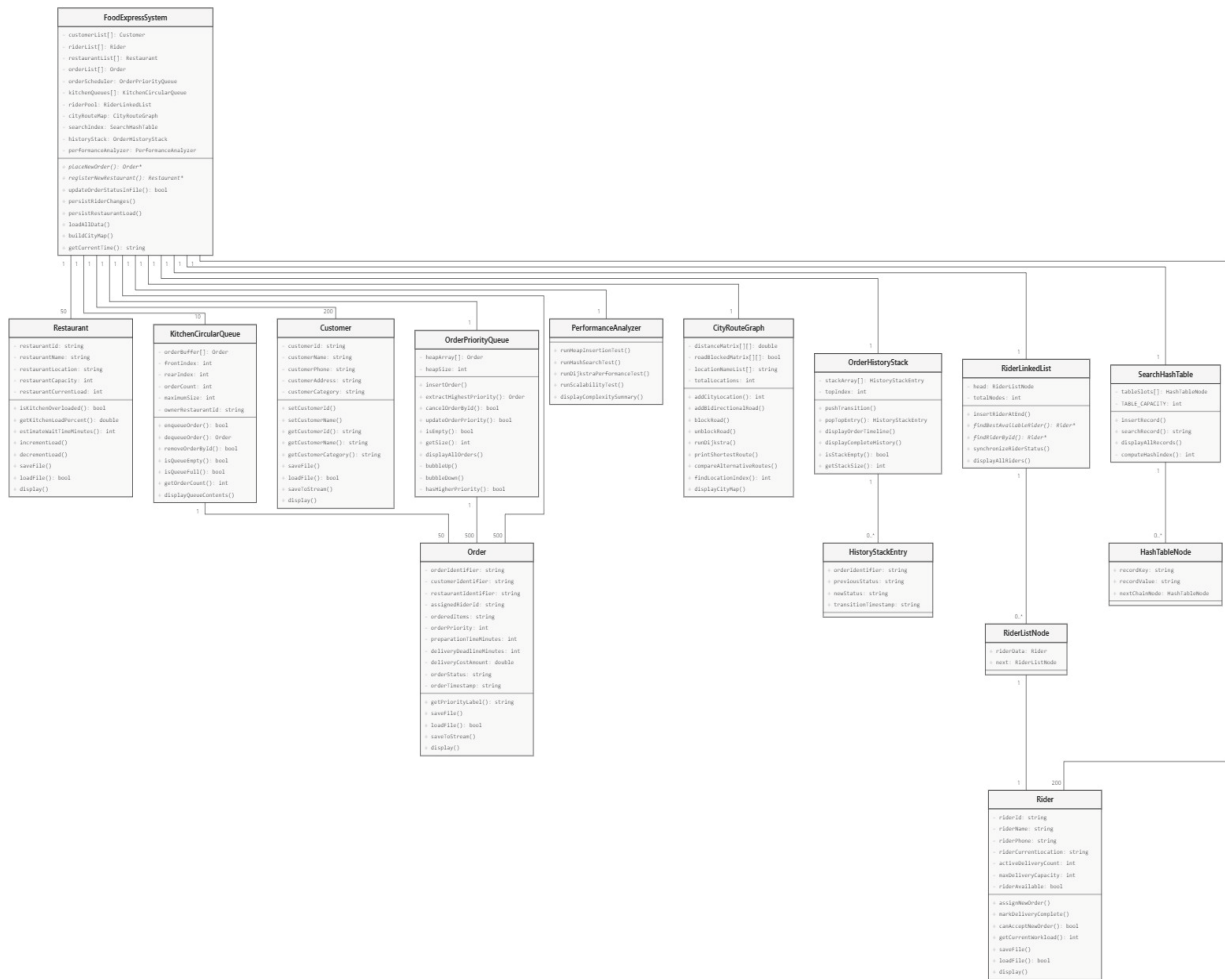
Layer	Component	Technology
Presentation	CLI Menu System (mainMenu + 9 module menus)	C++ cout/cin
Business Logic	FoodExpressSystem class	C++ OOP
Data Structures	Max-Heap, Circular Queue, Linked List, Graph, Hash Table, Stack	Custom C++ (No STL)
Persistence	7 .txt flat files	C++ fstream

1.2 Module-to-Data-Structure Mapping

Module	Data Structure	Class Name	Capacity
Dynamic Order Scheduling	Max-Heap	OrderPriorityQueue	500 orders
Kitchen Load Balancing	Circular Queue (×10)	KitchenCircularQueue	50 orders each
Rider Dispatch	Singly Linked List	RiderLinkedList	Dynamic
Route Optimization	Adjacency Matrix	CityRouteGraph	15 locations
Search & Retrieval	Hash Table (Chaining)	SearchHashTable	101 buckets
Order History	Stack	OrderHistoryStack	2000 entries
Performance Analysis	Algorithm Benchmark	PerformanceAnalyzer	N/A

2. Class Diagram & Relationships

The system is built around 13 custom classes. The FoodExpressSystem class acts as the facade — it composes all six data structures and manages the four entity collections. No inheritance hierarchy is used; the design favors composition over inheritance to keep each class independently testable.



2.1 Entity Classes

Class	Key Fields	Responsibility
Customer	customerId, customerName, customerPhone, customerAddress, customerCategory	Stores customer profile. Supports file persistence (saveFile / loadFile / saveToStream). Used to auto-determine order priority tier.
Rider	riderId, riderName, riderCurrentLocation, activeDeliveryCount, maxDeliveryCapacity, riderAvailable	Stores rider state. Exposes canAcceptNewOrder() and getCurrentWorkload() used by dispatch algorithm.
Restaurant	restaurantId, restaurantName, restaurantLocation, restaurantCapacity, restaurantCurrentLoad	Stores restaurant capacity state. Exposes isKitchenOverloaded(), getKitchenLoadPercent(), estimateWaitTimeMinutes().

Order	orderIdentifier, customerIdentifier, restaurantIdentifier, orderPriority, preparationTimeMinutes, deliveryCostAmount, orderStatus, orderTimestamp	Core data object. Carried through all modules — scheduled in heap, queued in kitchen, assigned to rider, tracked in history stack.
-------	---	--

2.2 Data Structure Classes

Class	Internal Structure	Key Methods
OrderPriorityQueue	Order heapArray[500]; int heapSize	insertOrder(), extractHighestPriority(), cancelOrderByld(), updateOrderPriority(), displayAllOrders()
KitchenCircularQueue	Order orderBuffer[CAPACITY]; int front, rear, count	enqueueOrder(), dequeueOrder(), removeOrderByld(), isQueueFull(), displayQueueContents()
RiderLinkedList	RiderListNode* head; int totalNodes	insertRiderAtEnd(), findBestAvailableRider(), findRiderByld(), synchronizeRiderStatus()
CityRouteGraph	double distanceMatrix[15][15]; bool roadBlockedMatrix[15][15]; string locationNameList[15]	runDijkstra(), printShortestRoute(), compareAlternativeRoutes(), blockRoad(), unblockRoad()
SearchHashTable	HashTableNode* tableSlots[101]	insertRecord(), searchRecord(), displayAllRecords()
OrderHistoryStack	HistoryStackEntry stackArray[2000]; int topIndex	pushTransition(), popTopEntry(), displayOrderTimeline(), displayCompleteHistory()
PerformanceAnalyzer	No persistent state	runHeapInsertionTest(), runHashSearchTest(), runDijkstraPerformanceTest(), runScalabilityTest(), displayComplexitySummary()

2.3 Class Relationships

Relationship	From	To	Type	Multiplicity
Composition	FoodExpressSystem	OrderPriorityQueue	owns	1 → 1
Composition	FoodExpressSystem	KitchenCircularQueue	owns	1 → 10
Composition	FoodExpressSystem	RiderLinkedList	owns	1 → 1
Composition	FoodExpressSystem	CityRouteGraph	owns	1 → 1
Composition	FoodExpressSystem	SearchHashTable	owns	1 → 1
Composition	FoodExpressSystem	OrderHistoryStack	owns	1 → 1
Composition	FoodExpressSystem	PerformanceAnalyzer	owns	1 → 1
Aggregation	FoodExpressSystem	Customer[]	manages	1 → 0..200
Aggregation	FoodExpressSystem	Rider[]	manages	1 → 0..200
Aggregation	FoodExpressSystem	Restaurant[]	manages	1 → 0..50
Aggregation	FoodExpressSystem	Order[]	manages	1 → 0..500
Composition	OrderPriorityQueue	Order	stores copies	1 → 0..500
Composition	KitchenCircularQueue	Order	stores copies	1 → 0..50
Composition	RiderLinkedList	RiderListNode	links	1 → 0..*
Composition	RiderListNode	Rider	wraps	1 → 1
Composition	OrderHistoryStack	HistoryStackEntry	stores	1 → 0..2000
Composition	SearchHashTable	HashTableNode	chains	1 → 0..*

3. Module Design

3.1 Module 1 — Dynamic Order Scheduling Engine

Data Structure: Max-Heap | **Class:** OrderPriorityQueue | **Capacity:** 500 orders

A Max-Heap provides $O(\log n)$ insertion and $O(\log n)$ extraction of the highest-priority order. This is optimal for a real-time scheduling engine where VIP orders must be processed before standard ones. A sorted linked list would give $O(n)$ extraction; a FIFO queue would completely ignore priority.

Priority Levels

Level	Label	Customer Tier	Processing Policy
4	CRITICAL	VIP	Always first
3	HIGH	Premium	Before Regular
2	MEDIUM	Regular	Default tier
1	LOW	Basic	Last processed

Supported Operations

Operation	Method	Complexity
Insert new order	insertOrder()	$O(\log n)$
Get highest priority order	extractHighestPriority()	$O(\log n)$
Cancel order by ID	cancelOrderById()	$O(n)$ scan + $O(\log n)$
Update order priority	updateOrderPriority()	$O(n)$ scan + $O(\log n)$
Peek top order	peekTopOrder()	$O(1)$
View all orders	displayAllOrders()	$O(n)$

3.2 Module 2 — Kitchen Load Balancing Module

Data Structure: Circular Queue | **Class:** KitchenCircularQueue | **Instances:** Up to 10

A Circular Queue models kitchen order flow — orders enter the rear and are prepared from the front. The circular design avoids wasted array space with $O(1)$ enqueue/dequeue. When a kitchen exceeds 80% load, the system detects overload and can rebalance by routing new orders to a less-loaded queue.

Operation	Method	Complexity
Enqueue order	enqueueOrder()	$O(1)$
Dequeue order	dequeueOrder()	$O(1)$
Remove by ID	removeOrderById()	$O(n)$
Detect overload	isKitchenOverloaded()	$O(1)$
Get load percent	getKitchenLoadPercent()	$O(1)$
Estimate wait time	estimateWaitTimeMinutes()	$O(1)$
Register restaurant	registerNewRestaurant()	$O(1)$

3.3 Module 3 — Rider Dispatch & Assignment Optimization

Data Structure: Singly Linked List | **Class:** RiderLinkedList | **Algorithm:** Greedy Best-Fit

A singly linked list is used for the rider pool because riders are dynamically added and the system only needs sequential traversal to find the best available rider. Pointer-based allocation avoids fixed array

sizing. The greedy best-fit algorithm selects the least-loaded eligible rider — minimizing per-rider delivery delay while balancing fleet workload.

Rider Selection Criteria (in order)

- riderAvailable == true
- activeDeliveryCount < maxDeliveryCapacity
- Among eligible: lowest getCurrentWorkload() wins

3.4 Module 4 — Real-Time Route Optimization

Data Structure: Adjacency Matrix | Algorithm: Dijkstra's $O(V^2)$ | $V = 15$ locations

An adjacency matrix is chosen because the city graph is dense (many roads between 15 fixed locations) and $V=15$ is small. Matrix gives $O(1)$ edge lookup. Dijkstra's algorithm (unvisited-set scan, $O(V^2)$) finds the optimal delivery route. A priority-queue Dijkstra ($O(E \log V)$) would only outperform at $V > 1000$.

City Locations (15 nodes)

Index	Location	Index	Location	Index	Location
0	Mall Road	5	Cantt	10	Shadman
1	Gulberg	6	Bahria Town	11	Ichra
2	DHA Phase 1	7	Wapda Town	12	Bhatta Chowk
3	Model Town	8	Faisal Town	13	Allama Iqbal Airport
4	Johar Town	9	Garden Town	14	Ring Road

Delivery Cost Formula: Cost (Rs.) = 50 + Distance(km) × 15

3.5 Module 5 — Search & Retrieval Engine

Data Structure: Hash Table (Separate Chaining) | Buckets: 101 | Hash: Polynomial Rolling

A hash table with chaining provides $O(1)$ average-case search — critical for a dispatch engine resolving thousands of ID lookups. A prime bucket count (101) minimizes hash collisions. Chaining handles collisions gracefully without clustering problems of open addressing.

Hash Function

```
hash = 0
for each character c in key:
    hash = (hash * 31 + ascii(c)) % 101
return hash
```

Indexed Entities

- Customers (prefix: C-)
- Riders (prefix: R-)
- Restaurants (custom ID)
- Orders (prefix: ORD-)

3.6 Module 6 — Order History & Tracking Module

Data Structure: Stack | **Class:** OrderHistoryStack | **Capacity:** 2000 transitions

A stack is the natural structure for undo/redo — LIFO ordering ensures the most recent status change is always at the top. Every status update pushes a HistoryStackEntry. The entire stack can be scanned to reconstruct any order's complete timeline. Undo pops the top and restores the previous status.

Tracked Status Transitions

new → **placed** → **accepted** → **queued** → **prepared** → **assigned** → **picked** → **delivered**

Any state → *delayed* / *cancelled* / *rerouted*

3.7 Module 7 — Performance Analysis Module

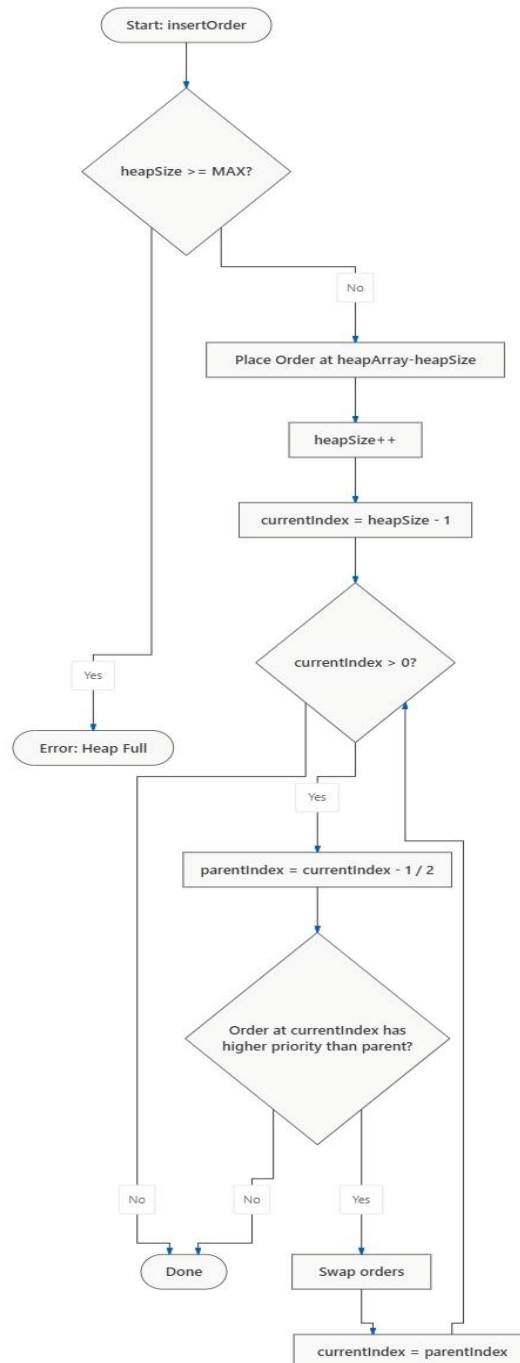
Class: PerformanceAnalyzer | **Tests:** Heap insertion, Hash search, Dijkstra, Scalability, Complexity

Test	What It Measures	Dataset Sizes
Heap Insertion Test	Actual insert time vs theoretical $O(\log n)$	10, 50, 100, 250, 500 orders
Hash Search Test	$O(1)$ avg vs $O(n)$ worst-case analysis	Load factor at 50%, 75%, 100%
Dijkstra Performance	Route calc time at different V sizes	V = 5, 10, 15 locations
Scalability Test	All structures under increasing N	N = 100, 250, 500 records
Complexity Summary	Theoretical Big-O table for all modules	All 6 data structures

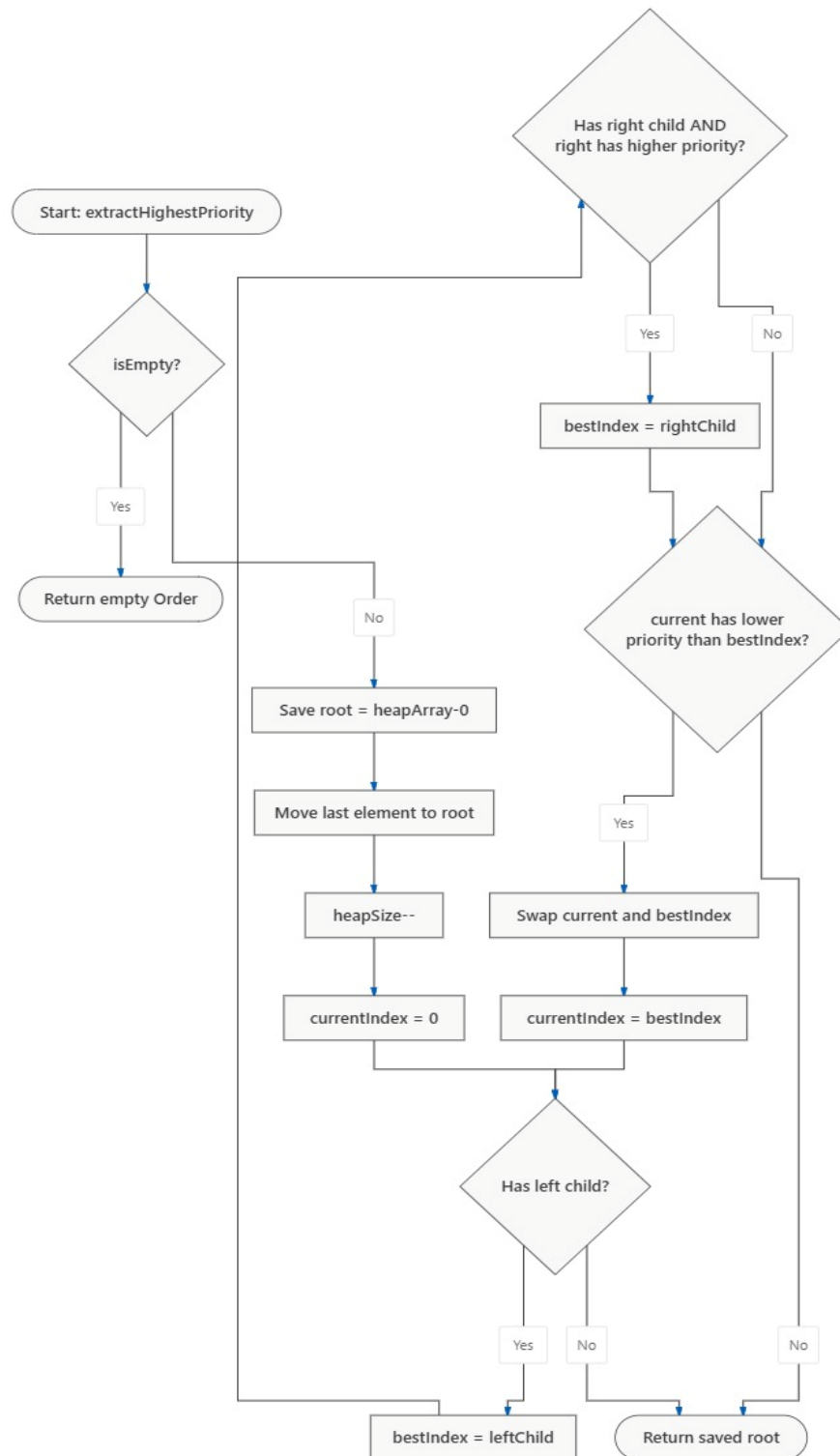
4. Algorithm Flowcharts

The following describes the step-by-step logic of each core algorithm. Mermaid diagram source is provided in the accompanying *UML_Design_Documentation.md*.

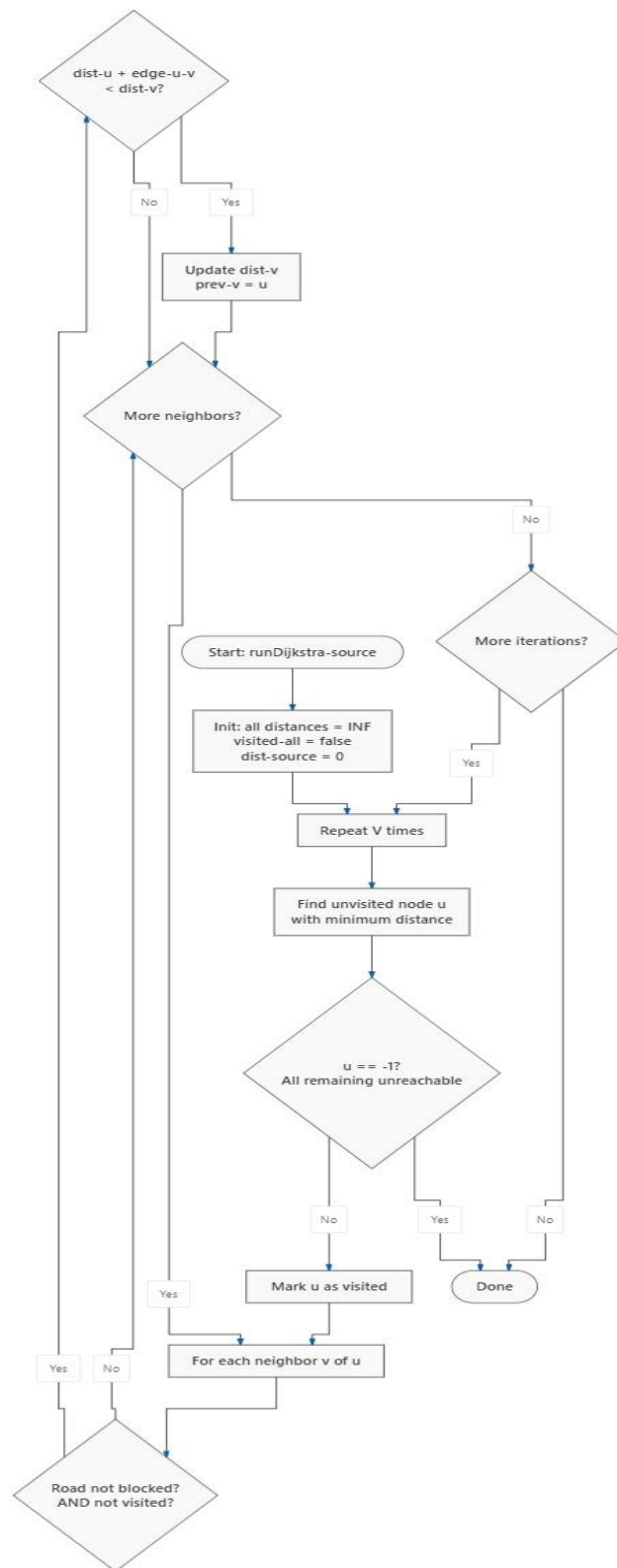
4.1 Max-Heap Insertion (bubbleUp)



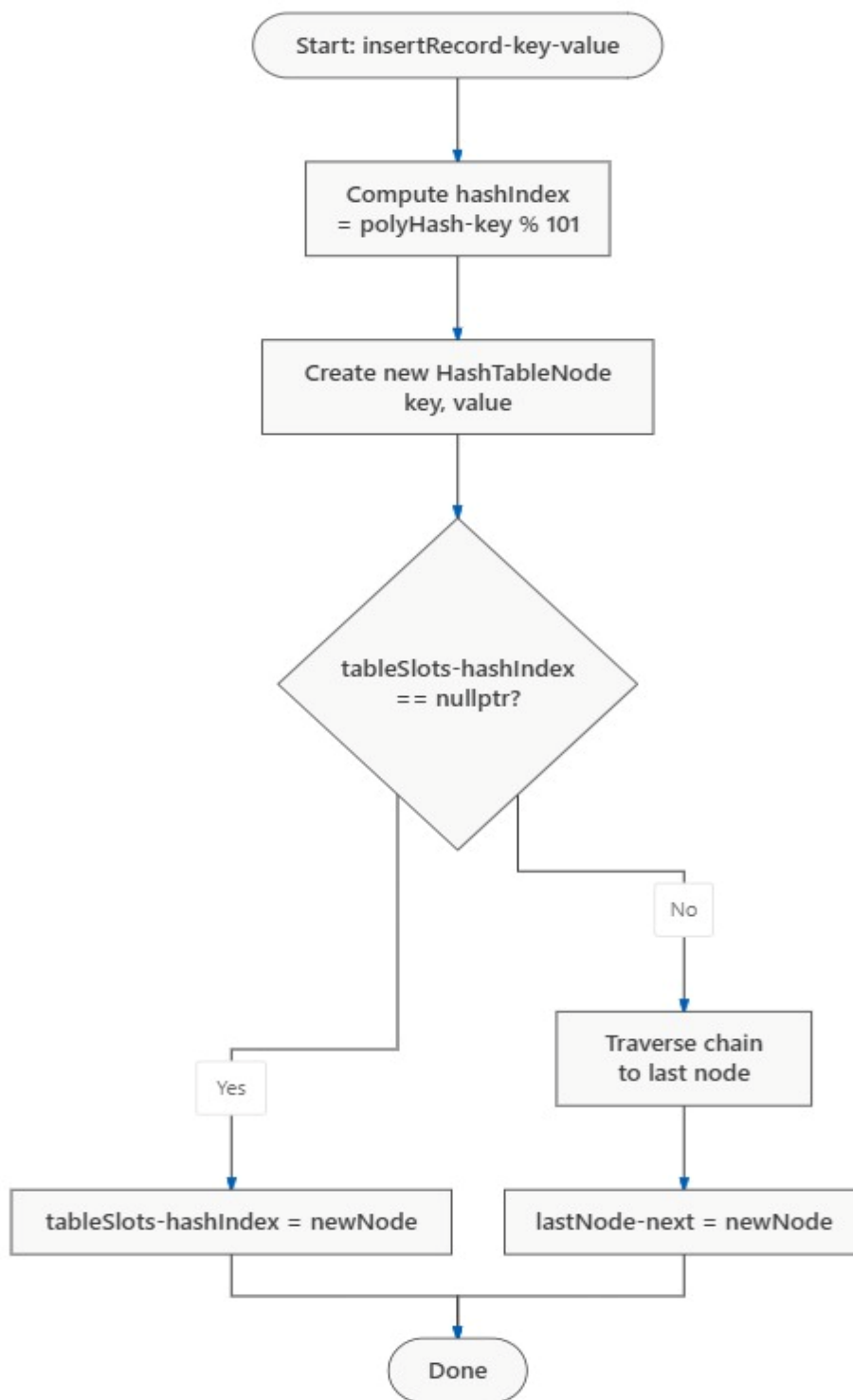
4.2 Max-Heap Extract Highest Priority (bubbleDown)



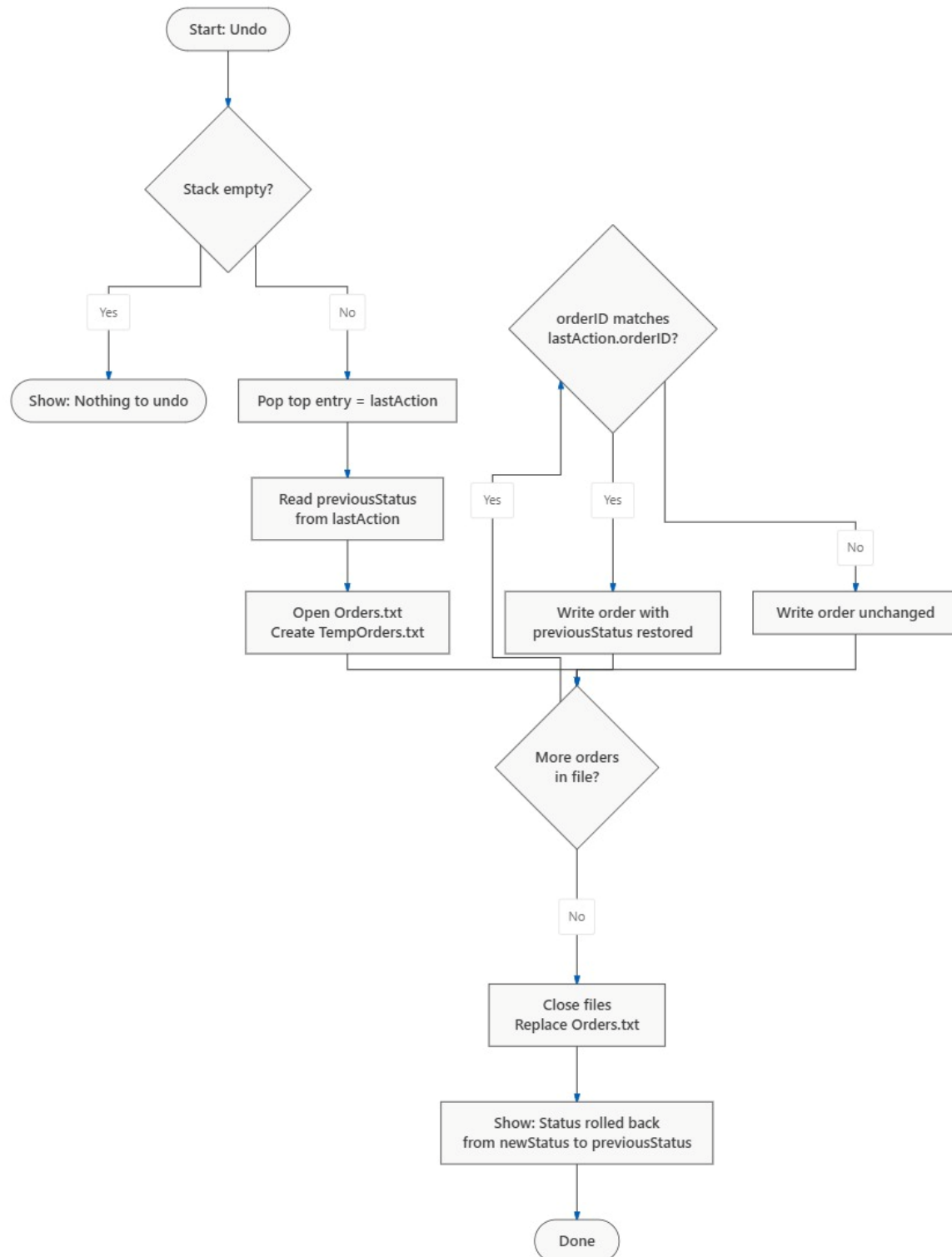
4.3 Dijkstra Shortest Path



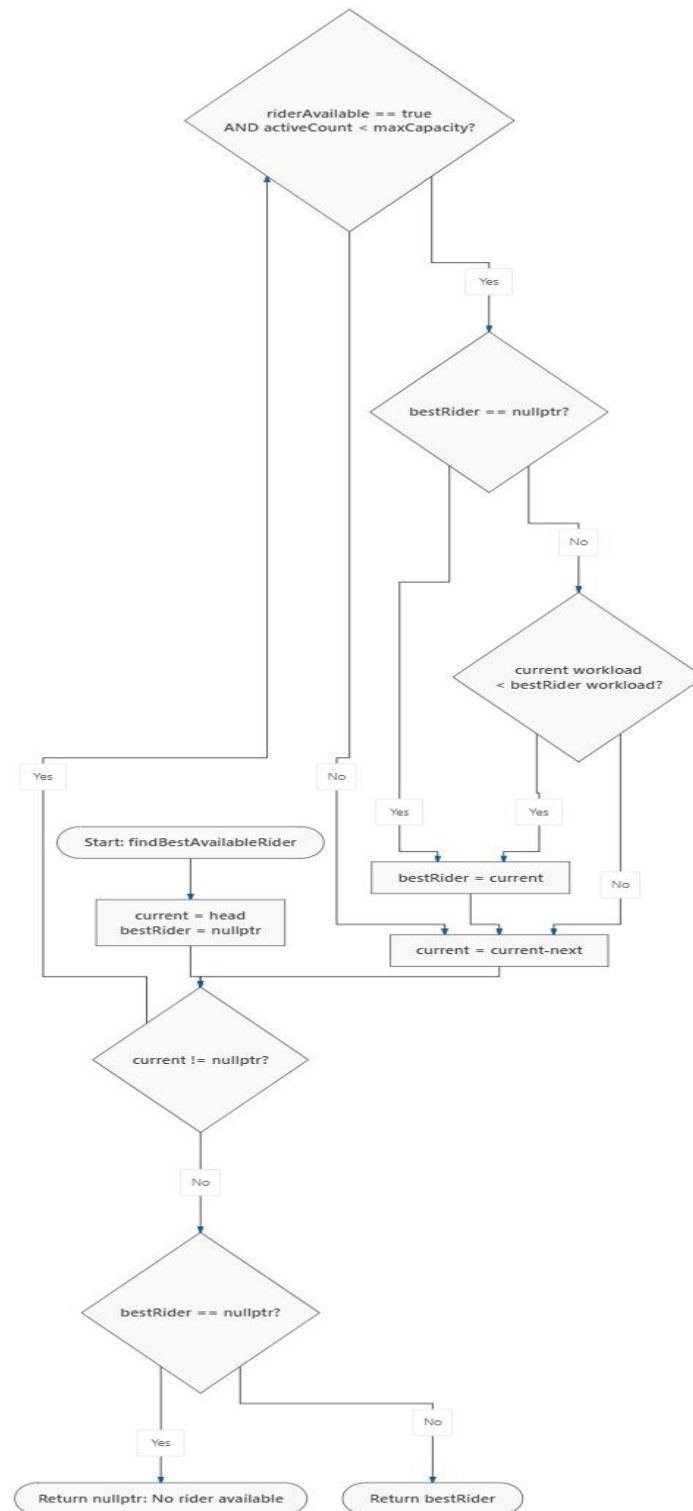
4.4 Hash Table Insert with Chaining



4.5 Order Status Undo Operation



4.6 Rider Best-Fit Selection



5. Complexity Analysis

5.1 Time Complexity

Module	Operation	Data Structure	Best	Average	Worst
Order Scheduling	Insert	Max-Heap	$O(1)$	$O(\log n)$	$O(\log n)$
Order Scheduling	Extract Max	Max-Heap	$O(\log n)$	$O(\log n)$	$O(\log n)$
Order Scheduling	Cancel / Update	Max-Heap	$O(n)$	$O(n)$	$O(n)$
Order Scheduling	Peek Top	Max-Heap	$O(1)$	$O(1)$	$O(1)$
Kitchen Load	Enqueue	Circular Queue	$O(1)$	$O(1)$	$O(1)$
Kitchen Load	Dequeue	Circular Queue	$O(1)$	$O(1)$	$O(1)$
Kitchen Load	Remove by ID	Circular Queue	$O(1)$	$O(n)$	$O(n)$
Rider Dispatch	Find Best Rider	Linked List	$O(1)$	$O(n)$	$O(n)$
Rider Dispatch	Insert Rider	Linked List	$O(1)$	$O(n)$	$O(n)$
Rider Dispatch	Find by ID	Linked List	$O(1)$	$O(n)$	$O(n)$
Route Optimization	Dijkstra	Adj Matrix	$O(V^2)$	$O(V^2)$	$O(V^2)$
Route Optimization	Edge lookup	Adj Matrix	$O(1)$	$O(1)$	$O(1)$
Route Optimization	Block/Unblock	Adj Matrix	$O(1)$	$O(1)$	$O(1)$
Search & Retrieval	Insert record	Hash Table	$O(1)$	$O(1)$	$O(n)$
Search & Retrieval	Search by ID	Hash Table	$O(1)$	$O(1)$	$O(n)$
Order History	Push transition	Stack	$O(1)$	$O(1)$	$O(1)$
Order History	Pop (Undo)	Stack	$O(1)$	$O(1)$	$O(1)$
Order History	View Timeline	Stack	$O(n)$	$O(n)$	$O(n)$

5.2 Space Complexity

Data Structure	Space Complexity	Fixed Capacity	Notes
Max-Heap	$O(n)$	500 orders	Static array allocation
Circular Queue ($\times 10$)	$O(k)$ per queue	50 orders / queue	k = queue capacity
Linked List	$O(n)$	Dynamic	Heap-allocated nodes
Adjacency Matrix	$O(V^2)$	$15 \times 15 = 225$ cells	Constant (V fixed)
Hash Table	$O(n + m)$	101 buckets	n =records, m =buckets
Stack	$O(n)$	2000 entries	Static array allocation

6. Ethical & Professional Practices

6.1 Customer Data Privacy

The system stores customer name, phone number, delivery address, order history, and customer category in plain-text .txt files. The following ethical safeguards are applied in this prototype:

- **Minimal Data Collection:** Only data required for order processing is stored. No payment card details or financial credentials are persisted.
- **Access Layer Control:** All file access is channelled through the FoodExpressSystem class — no module accesses files directly.
- **Data Retention Awareness:** A production deployment would enforce retention limits (e.g., auto-purge after 90 days per GDPR / PEC guidelines).
- **Anonymized Simulation:** The performance analysis module uses only synthetic dummy data — no real customer records are exposed during benchmarking.
- **No Authentication Bypass:** System is CLI-based and deployed locally — no network exposure in this lab prototype.

6.2 Responsible Rider Workload Management

- Each rider's active delivery count is capped at their declared maxDeliveryCapacity — preventing forced overloading.
- The greedy dispatcher always assigns the least-loaded eligible rider — distributing work fairly across the fleet.
- Rider availability flag allows riders to mark themselves unavailable without system interference.

6.3 Scalability Limits & Production Considerations

Resource	Prototype Limit	Production Recommendation
Max Customers	200 (static array)	Dynamic allocation / DB-backed
Max Riders	200 (static array)	Dynamic allocation
Max Restaurants	50 (static array)	Dynamic allocation
Max Orders	500 (static array)	Streaming / DB storage
History Stack	2000 entries (static)	File-backed append log
File Storage	.txt flat files	SQLite / PostgreSQL
Concurrency	Single-threaded	Thread-safe DS + mutex

6.4 Algorithm Design Justification

Design Choice	Alternative Considered	Why Our Choice Wins
Max-Heap for scheduling	Sorted linked list	$O(\log n)$ vs $O(n)$ extraction at scale
Circular Queue for kitchen	Regular array queue	$O(1)$ enqueue/dequeue, no wasted space, no shifting
Linked List for riders	Static array	Dynamic sizing; sequential traversal sufficient for small fleet
Adjacency Matrix + Dijkstra	Adjacency list + A*	Dense graph, $V=15$ is constant; matrix gives $O(1)$ edge lookup
Hash Table chaining	Open addressing	No clustering; handles load factor > 0.7 gracefully
Stack for history	Array with index pointer	Natural undo semantics; $O(1)$ push/pop; entire scan for timeline

6.5 Professional Engineering Standards

Modular Design: Each DSA module is encapsulated in its own class with a clear public interface — single responsibility principle enforced.

Defensive Coding: All file operations verify open success before reading/writing. Null pointer guards protect all dynamic dereferences.

No Memory Leaks: The FoodExpressSystem destructor explicitly frees all heap-allocated Customer, Rider, Restaurant, and Order objects.

Consistent Error Reporting: All failure paths output descriptive error messages to the user. No silent failures.

No STL Containers Used: All data structures (heap, queue, list, graph, hash table, stack) are fully custom-implemented per lab requirements.

Code Documentation: Each class and major method is preceded by a structured comment header explaining purpose, data structure used, and complexity.